

Additional NB comment resolutions for Kona 2025

Document #: P3923R0
Date: 2025-11-07
Project: Programming Language C++
Audience: LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

[1 Introduction](#)

[2 Wording](#)

[2.1 AT 7-213](#)

[2.2 US 140-233](#)

[2.3 US 141-235](#)

[2.4 US 145-234](#)

[2.5 US 147-240](#)

[2.6 US 164-203](#)

[2.7 US 126-189](#)

[2.8 US 227-346 and US 229-347](#)

[2.9 US 221-339](#)

[2.10 US 225-341](#)

[3 References](#)

§ 1 Introduction

This paper provides wording to resolve the following national body comments on the C++26 CD:

- [AT 7-213](#)
- [US 140-233](#)
- [US 141-235](#)
- [US 145-234](#)
- [US 147-240](#)
- [US 164-203](#)
- [US 126-189](#)
- [US 227-346](#)
- [US 229-347](#)
- [US 221-339](#)
- [US 225-341](#)

§ 2 Wording

This wording is relative to [\[N5014\]](#) except where noted.

§ 2.1 [AT 7-213](#)

Edit 22.4.7 [\[tuple.helper\]](#) as indicated:

```
template<class T> struct tuple_size<const T>;
```

4 [...]

5 [...]

6 In addition to being available via inclusion of the `<tuple>` header, the template is available when any of the headers `<array>`, `<complex>`, `<ranges>`, or `<utility>` are included.

```
template<size_t I, class T> struct tuple_element<I, const T>;
```

7 [...]

8 In addition to being available via inclusion of the `<tuple>` header, the template is available when any of the headers `<array>`, `<complex>`, `<ranges>`, or `<utility>` are included.

§ 2.2 [US 140-233](#)

[Drafting note: We add a cross-reference and a note to 23.3.9.1 [\[hive.overview\]](#) to clarify that “insertion” doesn’t include construction or assignment.]

1. Edit 23.3.9.1 [\[hive.overview\]](#) as indicated:

1 A hive is a type of sequence container that provides constant-time insertion and erasure operations. Storage is automatically managed in multiple memory blocks, referred to as *element blocks*. Insertion ([23.3.9.4 \[hive.modifiers\]](#)) position is determined by the container, and insertion may re-use the memory locations of erased elements. [*Note ?*: Construction and assignment are not considered to involve insertion operations. — *end note*]

2. Edit 23.3.9.2 [\[hive.cons\]](#) as indicated:

```
template<container-compatible-range<T> R>
    hive(from_range_t, R&& rg, const Allocator& = Allocator());
template<container-compatible-range<T> R>
    hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& =
        Allocator());
```

13 *Effects*: Constructs a hive object equal to with the elements of the range `rg`, using the specified allocator. If the second overload is called, also initializes *current-limits* with `block_limits`.

14 *Complexity*: Linear in `ranges::distance(rg)`.

```
hive(const hive& x);
hive(const hive& x, const type_identity_t<Allocator>& alloc);
```

15 *Preconditions:* T is *Cpp17CopyInsertable* into hive.

16 *Effects:* Constructs a hive object equal to with the elements of x. If the second overload is called, uses alloc. Initializes *current-limits* with x.*current-limits*.

17 *Complexity:* Linear in x.size().

```
hive(hive&& x) noexcept;
```

```
hive(hive&& x, const type_identity_t<Allocator>& alloc);
```

18 *Preconditions:* For the second overload, when allocator_traits<alloc>::is_always_equal::value is false, T meets the *Cpp17MoveInsertable* requirements.

19 *Effects:* When the first overload is called, or the second overload is called and alloc == x.get_allocator() is true, *current-limits* is set to x.*current-limits* and each element block is moved from x into *this. Pointers and references to the elements of x now refer to those same elements but as members of *this. Iterators referring to the elements of x will continue to refer to their elements, but they now behave as iterators into *this. If the second overload is called and alloc == x.get_allocator() is false, each element in x is moved into *this. References, pointers and iterators referring to the elements of x, as well as the past-the-end iterator of x, are invalidated.

20 *Postconditions:* x.empty() is true. The relative order of the elements of *this is the same as that of the elements of x prior to this call.

21 *Complexity:* If the second overload is called and alloc == x.get_allocator() is false, linear in x.size(). Otherwise constant.

```
hive(initializer_list<T> il, const Allocator& = Allocator());
```

```
hive(initializer_list<T> il, hive_limits block_limits, const Allocator& =  
    Allocator());
```

22 *Preconditions:* T is *Cpp17CopyInsertable* into hive.

13 *Effects:* Constructs a hive object equal to with the elements of il, using the specified allocator. If the second overload is called, also initializes *current-limits* with block_limits.

14 *Complexity:* Linear in il.size().

```
hive& operator=(const hive& x);
```

15 *Preconditions:* T is *Cpp17CopyInsertable* into hive and *Cpp17CopyAssignable*.

16 *Effects:* All elements in *this are either copy-assigned to, or destroyed. All elements in x are copied into *this, maintaining their relative order.

[Note 1: *current-limits* is unchanged. — end note]

17 *Complexity:* Linear in size() + x.size().

```
hive& operator=(hive&& x)
```

```
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
```

```

        allocator_traits<Allocator>::is_always_equal::value);
18  Preconditions: When

        (allocator_traits<Allocator>::propagate_on_container_move_assignment::value
         ||
         allocator_traits<Allocator>::is_always_equal::value)

is false, T is Cpp17MoveInsertable into hive and Cpp17MoveAssignable.

19  Effects: Each element in *this is either move-assigned to, or destroyed. When

        (allocator_traits<Allocator>::propagate_on_container_move_assignment::value
         ||
         get_allocator() == x.get_allocator())

is true, current-limits is set to x.current-limits and each element block is
moved from x into *this. Pointers and references to the elements of x now refer to
those same elements but as members of *this. Iterators referring to the elements of x
will continue to refer to their elements, but they now behave as iterators into *this, not
into x. When

        (allocator_traits<Allocator>::propagate_on_container_move_assignment::value
         ||
         get_allocator() == x.get_allocator())

is false, each element in x is moved into *this. References, pointers and iterators
referring to the elements of x, as well as the past-the-end iterator of x, are invalidated.

20  Postconditions: x.empty() is true. The relative order of the elements of *this is the
same as that of the elements of x prior to this call.

21  Complexity: Linear in size(). If

        (allocator_traits<Allocator>::propagate_on_container_move_assignment::value
         ||
         get_allocator() == x.get_allocator())

is false, also linear in x.size().

```

§ 2.3 US 141-235

[Drafting note: LWG concluded that the wording is already clear that the order does not change because nothing in the wording gives permission for it to change. However, it was pointed out that the first sentence of the *Remarks*: paragraph is similarly a “nothing happens” sentence and LWG decided to strike it to avoid the erroneous implication that “everything not explicitly described as not changing can change”.]

Edit 23.3.9.3 [[hive.capacity](#)] as indicated:

```
void reserve(size_type n);
```

3 *Effects:* [...]

4 *Postconditions:* [...]

- 5 *Throws:* [...]
- 6 *Complexity:* [...]
- 7 *Remarks:* **The size of the sequence is not changed.** All references, pointers, and iterators referring to elements in **this*, as well as the past-the-end iterator, remain valid.

§ 2.4 [US 145-234](#)

Edit 23.3.9.3 [\[hive.capacity\]](#) as indicated:

```
void trim_capacity() noexcept;
void trim_capacity(size_type n) noexcept;
```

- 12 *Effects:* For the first overload, all reserved blocks are deallocated, and `capacity()` is reduced accordingly. For the second overload, **if `n >= capacity()` is `true`, there are no effects; otherwise, `capacity()` is reduced to no less than `n`.**
- 13 *Complexity:* Linear in the number of reserved blocks deallocated.
- 14 *Remarks:* All references, pointers, and iterators referring to elements in **this*, as well as the past-the-end iterator, remain valid.

§ 2.5 [US 147-240](#)

[Drafting note: LWG determined that there is no good reason to check the block limits during or after the splice and therefore we can offer the strong exception guarantee.]

Edit 23.3.9.5 [\[hive.operations\]](#) as indicated:

```
void splice(hive& x);
void splice(hive&& x);
```

- 2 *Preconditions:* `get_allocator() == x.get_allocator()` is `true`.
- 3 *Effects:* If `addressof(x) == this` is `true`, the behavior is erroneous and there are no effects. **If an exception is thrown, there are no effects.** Otherwise, inserts the contents of `x` into **this* and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of **this*. Iterators referring to the moved elements continue to refer to their elements, but they now behave as iterators into **this*, not into `x`.
- 4 *Throws:* `length_error` if any of `x`'s active blocks are not within the bounds of *current-limits*.
- 5 *Complexity:* Linear in the sum of all element blocks in `x` plus all element blocks in **this*.
- 6 *Remarks:* Reserved blocks in `x` are not transferred into **this*. If `addressof(x) == this` is `false`, invalidates the past-the-end iterator for both `x` and **this*.

§ 2.6 [US 164-203](#)

1. Edit 26.8.7.3 [\[set.union\]](#) as indicated:

```

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_union(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              ForwardIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class
        Compare>
constexpr OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_union(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
        weakly_increamentable 0, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
constexpr ranges::set_union_result<I1, I2, 0>
    ranges::set_union(I1 first1, S1 last1, I2 first2, S2 last2, 0 result, Comp
                      comp = {},
                      Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_increamentable 0,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 =
        identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, 0, Comp, Proj1, Proj2>
constexpr ranges::set_union_result<borrowed_iterator_t<R1>,
        borrowed_iterator_t<R2>, 0>
    ranges::set_union(R1&& r1, R2&& r2, 0 result, Comp comp = {},
                      Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1>
        S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator 0, sized_sentinel_for<0> OutS, class Comp =
        ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
ranges::set_union_result<I1, I2, 0>
    ranges::set_union(Ep&& exec, I1 first1, S1 last1,
                      I2 first2, S2 last2, 0 result, OutS result_last,
                      Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized-random-access-range R1, sized-random-access-
        range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>

```

```

requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp,
    Proj1, Proj2>
ranges::set_union_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
    borrowed_iterator_t<OutR>>
    ranges::set_union(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp =
        {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — `comp` be `less{}`, and `proj1` and `proj2` be `identity{}` for the overloads with no parameters by those names;
- (1.2) — M be the number of elements in the sorted union (see below) `last1 - first1` plus the number of elements in `[first2, last2)` that are not present in `[first1, last1)`;
- (1.3) — `result_last` be `result + M` for the overloads with no parameter `result_last` or `result_r`;
- (1.4) — N be $\min(M, \text{result_last} - \text{result})$.

2 *Preconditions:* The ranges `[first1, last1)` and `[first2, last2)` are sorted with respect to `comp` and `proj1` or `proj2`, respectively. The resulting range does not overlap with either of the original ranges.

3 *Effects:* Constructs a sorted union of N elements from the two ranges; that is, the set of elements that are present in one or both of the ranges. If `[first1, last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, then all m elements from the first range are included in the union, in order, and then the final $\max(n - m, 0)$ elements from the second range are included in the union, in order. If, of those elements, k elements from the first range are copied to the output range, then the first $\min(k, n)$ elements from the second range are considered *skipped*. Copies the first N elements of the sorted union to the range `[result, result + N)`.

4 *Returns:*

- (4.1) — `result_last` for the overloads in namespace `std`.
- (4.2) — `{last1, last2, result + N}` for the overloads in namespace `ranges`, if N is equal to M .
- (4.3) — Otherwise, `{j1, j2, result_last}` `{first1 + A, first2 + B, result_last}` for the overloads in namespace `ranges`, where the iterators `j1` and `j2` point to positions past the last A and B are the numbers of copied or skipped elements in `[first1, last1)` and `[first2, last2)`, respectively.

5 *Complexity:* At most $2 * ((\text{last1} - \text{first1}) + (\text{last2} - \text{first2})) - 1$ comparisons and applications of each projection.

6 *Remarks:* Stable (16.4.6.8 [\[algorithm.stable\]](#)). If `[first1, last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, then all m elements from the first range are copied to the

output range, in order, and then the final $\max(n - m, 0)$ elements from the second range are copied to the output range, in order.

2. Edit 26.8.7.4 [\[set.intersection\]](#) as indicated:

```
template<class InputIterator1, class InputIterator2,
        class OutputIterator>
constexpr OutputIterator
    set_intersection(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, InputIterator2 last2,
                    OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_intersection(ExecutionPolicy&& exec,
                    ForwardIterator1 first1, ForwardIterator1 last1,
                    ForwardIterator2 first2, ForwardIterator2 last2,
                    ForwardIterator result);

template<class InputIterator1, class InputIterator2,
        class OutputIterator, class Compare>
constexpr OutputIterator
    set_intersection(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, InputIterator2 last2,
                    OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_intersection(ExecutionPolicy&& exec,
                    ForwardIterator1 first1, ForwardIterator1 last1,
                    ForwardIterator2 first2, ForwardIterator2 last2,
                    ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr ranges::set_intersection_result<I1, I2, O>
    ranges::set_intersection(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 =
        identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr ranges::set_intersection_result<borrowed_iterator_t<R1>,
        borrowed_iterator_t<R2>, O>
    ranges::set_intersection(R1&& r1, R2&& r2, O result,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1>
        S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp =
        ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_intersection_result<I1, I2, O>
    ranges::set_intersection(Ep&& exec, I1 first1, S1 last1,
```



```

        I2 first2, S2 last2, 0 result, OutS result_last,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution-policy Ep, sized-random-access-range R1, sized-random-access-
range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp,
        Proj1, Proj2>
ranges::set_intersection_result<borrowed_iterator_t<R1>,
        borrowed_iterator_t<R2>,
        borrowed_iterator_t<OutR>>
ranges::set_intersection(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp
        comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — comp be less`{}`, and proj1 and proj2 be identity`{}` for the overloads with no parameters by those names;
- (1.2) — M be the number of elements in the sorted intersection (see below) `[first1, last1)` that are present in `[first2, last2)`;
- (1.3) — result_last be result + M for the overloads with no parameter result_last or result_r;
- (1.4) — N be $\min(M, \text{result_last} - \text{result})$.

2 *Preconditions:* The ranges `[first1, last1)` and `[first2, last2)` are sorted with respect to comp and proj1 or proj2, respectively. The resulting range does not overlap with either of the original ranges.

3 *Effects:* Constructs a sorted intersection of the N elements from the two ranges; that is, the set of elements that are present in both of the ranges. If `[first1, last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, the first $\min(m, n)$ elements from the first range are included in the sorted intersection. If, of those elements, k elements from the first range are copied to the output range, then the first k elements from the second range are considered *skipped*. If $N < M$, a non-copied element is also considered skipped if it compares less than the $(N + 1)^{\text{th}}$ element of the sorted intersection. Copies the first N elements of the sorted intersection to the range `[result, result + N)`.

4 *Returns:*

- (4.1) — result_last for the overloads in namespace std.
- (4.2) — `{last1, last2, result + N}` for the overloads in namespace ranges, if N is equal to M .
- (4.3) — Otherwise, `{j1, j2, result_last}` `{first1 + A, first2 + B, result_last}` for the overloads in namespace ranges, where the iterators j1 and j2 point to positions past the last A and B are the numbers of copied or skipped elements in `[first1, last1)` and `[first2, last2)`, respectively.

5 *Complexity:* At most $2 * ((\text{last1} - \text{first1}) + (\text{last2} - \text{first2})) - 1$ comparisons and applications of each projection.

- 6 *Remarks:* Stable (16.4.6.8 [\[algorithm.stable\]](#)). If $[first1, last1)$ contains m elements that are equivalent to each other and $[first2, last2)$ contains n elements that are equivalent to them, the first $\min(m, n)$ elements are copied from the first range to the output range, in order.

3. Edit 26.8.7.5 [\[set.difference\]](#) as indicated:

```
template<class InputIterator1, class InputIterator2,
        class OutputIterator>
constexpr OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, InputIterator2 last2,
                  OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_difference(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  ForwardIterator result);

template<class InputIterator1, class InputIterator2,
        class OutputIterator, class Compare>
constexpr OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, InputIterator2 last2,
                  OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_difference(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
        weakly_incrementable 0, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<I1, 0>
    ranges::set_difference(I1 first1, S1 last1, I2 first2, S2 last2, 0 result,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incrementable 0,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 =
        identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, 0, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<borrowed_iterator_t<R1>, 0>
    ranges::set_difference(R1&& r1, R2&& r2, 0 result,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1>
        S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator 0, sized_sentinel_for<0> OutS, class Comp =
        ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
```

```

ranges::set_difference_result<I1, 0>
    ranges::set_difference(Ep&& exec, I1 first1, S1 last1,
                          I2 first2, S2 last2, 0 result, OutS result_last,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution-policy Ep, sized-random-access-range R1, sized-random-access-
    range R2,
    sized-random-access-range OutR, class Comp = ranges::less,
    class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp,
    Proj1, Proj2>
ranges::set_difference_result<borrowed_iterator_t<R1>,
    borrowed_iterator_t<OutR>>
    ranges::set_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp
        = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — comp be less`{}`, and proj1 and proj2 be identity`{}` for the overloads with no parameters by those names;
- (1.2) — M be the number of elements in the sorted difference (see below) `[first1, last1)` that are not present in `[first2, last2)`;
- (1.3) — result_last be result + M for the overloads with no parameter result_last or result_r;
- (1.4) — N be $\min(M, \text{result_last} - \text{result})$.

2 *Preconditions:* The ranges `[first1, last1)` and `[first2, last2)` are sorted with respect to comp and proj1 or proj2, respectively. The resulting range does not overlap with either of the original ranges.

3 ~~*Effects:* Copies N elements of the range `[first1, last1)` which are not present in the range `[first2, last2)` to the range `[result, result + N)`. The elements in the constructed range are sorted.~~

3 *Effects:* Constructs a sorted difference between the elements from the two ranges; that is, the set of elements that are present in the range `[first1, last1)` but not `[first2, last2)`. If `[first1, last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, the last $\max(m - n, 0)$ elements from `[first1, last1)` are included in the sorted difference, in order. Copies the first N elements of the sorted difference to the range `[result, result + N)`.

4 *Returns:*

- (4.1) — result_last for the overloads in namespace std.
- (4.2) — `{last1, result + N}` for the overloads in namespace ranges, if N is equal to M .
- (4.3) — Otherwise, `{j1, result_last}` for the overloads in namespace ranges, where the iterator j1 points to the position of the past the last copied or skipped element in `[first1, last1)` corresponding to the $(N + 1)^{th}$ element of the sorted difference.

- 5 Complexity: At most $2 * ((\text{last1} - \text{first1}) + (\text{last2} - \text{first2})) - 1$ comparisons and applications of each projection.
- 6 Remarks: Stable (16.4.6.8 [\[algorithm.stable\]](#)). If $[\text{first1}, \text{last1})$ contains m elements that are equivalent to each other and $[\text{first2}, \text{last2})$ contains n elements that are equivalent to them, the last $\max(m - n, 0)$ elements are copied from $[\text{first1}, \text{last1})$ to the output range, in order.

4. Edit 26.8.7.6 [\[set.symmetric.difference\]](#) as indicated:

```
template<class InputIterator1, class InputIterator2,
        class OutputIterator>
constexpr OutputIterator
    set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
                            InputIterator2 first2, InputIterator2 last2,
                            OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_symmetric_difference(ExecutionPolicy&& exec,
                            ForwardIterator1 first1, ForwardIterator1 last1,
                            ForwardIterator2 first2, ForwardIterator2 last2,
                            ForwardIterator result);

template<class InputIterator1, class InputIterator2,
        class OutputIterator, class Compare>
constexpr OutputIterator
    set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
                            InputIterator2 first2, InputIterator2 last2,
                            OutputIterator result, Compare comp);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_symmetric_difference(ExecutionPolicy&& exec,
                            ForwardIterator1 first1, ForwardIterator1 last1,
                            ForwardIterator2 first2, ForwardIterator2 last2,
                            ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
        weakly_increamentable 0, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
constexpr ranges::set_symmetric_difference_result<I1, I2, 0>
    ranges::set_symmetric_difference(I1 first1, S1 last1, I2 first2, S2 last2, 0
    result,
                                    Comp comp = {}, Proj1 proj1 = {},
                                    Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_increamentable 0,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 =
    identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, 0, Comp, Proj1, Proj2>
constexpr ranges::set_symmetric_difference_result<borrowed_iterator_t<R1>,
        borrowed_iterator_t<R2>, 0>
    ranges::set_symmetric_difference(R1&& r1, R2&& r2, 0 result, Comp comp = {},
    Proj1 proj1 = {}, Proj2 proj2 = {});
```

```

template<execution-policy Ep, random_access_iterator I1, sized_sentinel_for<I1>
    S1,
    random_access_iterator I2, sized_sentinel_for<I2> S2,
    random_access_iterator OutS, sized_sentinel_for<OutS> OutS, class Comp =
    ranges::less,
    class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
ranges::set_symmetric_difference_result<I1, I2, 0>
ranges::set_symmetric_difference(Ep&& exec, I1 first1, S1 last1,
                                I2 first2, S2 last2, 0 result, OutS
                                result_last,
                                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2
                                = {});
template<execution-policy Ep, sized-random-access-range R1, sized-random-access-
    range R2,
    sized-random-access-range OutR, class Comp = ranges::less,
    class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp,
    Proj1, Proj2>
ranges::set_symmetric_difference_result<borrowed_iterator_t<R1>,
    borrowed_iterator_t<R2>,
    borrowed_iterator_t<OutR>>
ranges::set_symmetric_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r,
    Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2
    = {});

```

1 Let:

- (1.1) — comp be less`{}`, and proj1 and proj2 be identity`{}` for the overloads with no parameters by those names;
- (1.2) — ~~K be the number of elements in $[first1, last1)$ that are not present in $[first2, last2)$;~~
- (1.3) — M be the number of elements in the sorted symmetric difference (see below) $[first2, last2)$ that are not present in $[first1, last1)$;
- (1.4) — result_last be result + $M + K$ for the overloads with no parameter result_last or result_r;
- (1.5) — N be $\min(K + M, result_last - result)$.

2 *Preconditions:* The ranges $[first1, last1)$ and $[first2, last2)$ are sorted with respect to comp and proj1 or proj2, respectively. The resulting range does not overlap with either of the original ranges.

3 ~~*Effects:* Copies the elements of the range $[first1, last1)$ that are not present in the range $[first2, last2)$, and the elements of the range $[first2, last2)$ that are not present in the range $[first1, last1)$ to the range $[result, result + N)$. The elements in the constructed range are sorted.~~

3 *Effects:* Constructs a sorted symmetric difference of the elements from the two ranges; that is, the set of elements that are present in exactly one of $[first1, last1)$ and $[first2, last2)$. If $[first1, last1)$ contains m elements that are equivalent to each other and $[first2, last2)$ contains n elements that are equivalent to them, then $|m - n|$ of those elements are included in the symmetric difference: the last $m - n$ of these elements from $[first1, last1)$, in order, if $m > n$, and the last $n - m$ of these

elements from $[first2, last2)$, in order, if $m < n$. If $N < M$, a non-copied element is considered *skipped* if it compares less than or equivalent to the $(N + 1)^{th}$ element of the sorted symmetric difference, unless it is from the same range as that element and does not precede it. Copies the first N elements of the sorted symmetric difference to the range $[result, result + N)$.

4 *Returns:*

- (4.1) — `result_last` for the overloads in namespace `std`.
- (4.2) — $\{last1, last2, result + N\}$ for the overloads in namespace `ranges`, if N is equal to $M + K$.
- (4.3) — Otherwise, $\{j1, j2, result_last\}$ $\{first1 + A, first2 + B, result_last\}$ for the overloads in namespace `ranges`, where the iterators `j1` and `j2` point to positions past the last A and B are the numbers of copied or skipped elements in $[first1, last1)$ and $[first2, last2)$, respectively.

5 *Complexity:* At most $2 * ((last1 - first1) + (last2 - first2)) - 1$ comparisons and applications of each projection.

6 *Remarks:* Stable (16.4.6.8 [\[algorithm.stable\]](#)). If $[first1, last1)$ contains m elements that are equivalent to each other and $[first2, last2)$ contains n elements that are equivalent to them, then $|m - n|$ of those elements shall be copied to the output range: the last $m - n$ of these elements from $[first1, last1)$ if $m > n$, and the last $n - m$ of these elements from $[first2, last2)$ if $m < n$. In either case, the elements are copied in order.

§ 2.7 [US 126-189](#)

Edit 21.4.16 [\[meta.reflection.define.aggregate\]](#) as indicated:

```
template<reflection_range R = initializer_list<info>>
constexpr info define_aggregate(info class_type, R&& mdescrs);
```

7 Let C be the class represented by `class_type` and r_K be the K^{th} reflection value in `mdescrs`. For every r_K in `mdescrs`, let $(T_K, N_K, A_K, W_K, NUA_K)$ be the corresponding data member description represented by r_K .

8 *Constant When:*

- (8.1) — C is incomplete from every point in the evaluation context; [*Note 4:* C can be a class template specialization for which there is a reachable definition of the class template. In this case, the injected declaration is an explicit specialization. — *end note*]
- (8.2) — $[...]$
- (8.3) — $[...]$
- (8.4) — $[...]$

9 *Effects:* Produces an injected declaration D (7.7 [\[expr.const\]](#)) that defines C and has properties as follows:

- (9.1) — [...]
- (9.2) — [...]
- (9.3) — [...]
- (9.4) — If C is a specialization of a templated class T , and C is not a local class, then D is an explicit specialization of T .
- (9.5) — [...]
- (9.6) — [...]
- 10 Returns: `class_type`.

? Remarks: If C is a specialization of a templated class and it has not been instantiated, C is treated as an explicit specialization.

§ 2.8 US 227-346 and US 229-347

1. Edit 33.9.12.18 [\[exec.spawn.future\]](#) as indicated:

7 Let `spawn-future-state` be the exposition-only class template:

```
namespace std::execution {
    template<class Alloc, scope_token Token, sender Sender, class Env>
    struct spawn-future-state //
        exposition only
    : spawn-future-state-base<completion_signatures_of_t<future-spawned-
        sender<Sender, Env>>> {
    using sigs-t = //
        exposition only
        completion_signatures_of_t<future-spawned-sender<Sender, Env>>;
    using receiver-t = //
        exposition only
        spawn-future-receiver<sigs-t>;
    using op-t = //
        exposition only
        connect_result_t<future-spawned-sender<Sender, Env>, receiver-t>;

    spawn-future-state(Alloc alloc, Sender&& sndr, Token token, Env env) //
        exposition only
    : alloc(std::move(alloc)),
      op(connect(
          write_env(stop-when(std::forward<Sender>(sndr), ss.source.get_token()),
            std::move(env)),
          receiver-t(this))),
      token(std::move(token)),
      associated(token.try_associate()) {
        if (associated)
            start(op);
        else
            set_stopped(receiver-t(this));
    }

    void complete() noexcept override; //
        exposition only
    void consume(receiver auto& rcvr) noexcept; //
        exposition only
}
```

```

    void abandon() noexcept; //
        exposition only

private:
    using alloc-t = //
        exposition only
        allocator_traits<Alloc>::template rebind_alloc<spawn-future-state>;

    Alloc alloc-t alloc;
        // exposition only
    ssource-t ssource; //
        exposition only
    op-t op; //
        exposition only
    Token token; //
        exposition only
    bool associated; //
        exposition only

    void destroy() noexcept; //
        exposition only
};

template<class Alloc, scope_token Token, sender Sender, class Env>
spawn-future-state(Alloc alloc, Sender&& sndr, Token token, Env env) -> spawn-
future-state<Alloc, Token, Sender, Env>;
}

```

[...]

```
void destroy() noexcept;
```

12 Effects: Equivalent to:

```

    auto token = std::move(this->token);
    bool associated = this->associated;

    {
        - auto alloc = std::move(this->alloc);
        -
        - allocator_traits<alloc-t>::destroy(alloc, this);
        - allocator_traits<alloc-t>::deallocate(alloc, this, 1);

        + using traits = allocator_traits<Alloc>::template
            rebind_traits<spawn-future-state>;
        + typename traits::allocator_type alloc(std::move(this-
            >alloc));
        + traits::destroy(alloc, this);
        + traits::deallocate(alloc, this, 1);
    }

    if (associated)
        token.disassociate();

```

[...]

16 The expression `spawn_future(sndr, token, env)` has the following effects:

- (16.1) — Uses `alloc` to allocate and construct an object `s` of a type that is a specialization of `spawn-future-state` type `decltype(spawn-future-state(alloc, token.wrap(sndr), token, senv))` from `alloc`, `token.wrap(sndr)`, `token`, and `senv`. If an exception is thrown then any constructed objects are destroyed and any allocated memory is deallocated.
- (16.2) — Constructs an object `u` of a type that is a specialization of `unique_ptr` such that:
 - (16.2.1) — `u.get()` is equal to the address of `s`, and
 - (16.2.2) — `u.get_deleter()(u.release())` is equivalent to `u.release()->abandon()`.
- (16.3) — Returns `make-sender(spawn_future, std::move(u))`.

2. Edit 33.9.13.3 [\[exec.spawn\]](#) as indicated:

5 Let `spawn-state` be the exposition-only class template:

```
namespace std::execution {
    template<class Alloc, scope_token Token, sender Sender>
    struct spawn-state : spawn-state-base {           // exposition only
        using op-t = connect_result_t<Sender, spawn-receiver>; // exposition only

        spawn-state(Alloc alloc, Sender&& sndr, Token token); // exposition only
        void complete() noexcept override;                 // exposition only
        void run();                                         // exposition only

    private:
        using alloc-t =                                // exposition only
            allocator_traits<Alloc>::template rebind_alloc<spawn-state>;

        Alloc alloc-t alloc;                             // exposition
        only
        op-t op;                                           // exposition only
        Token token;                                       // exposition only

        void destroy() noexcept;                          // exposition only
    };
}
```

[...]

```
void destroy() noexcept;
```

9 *Effects*: Equivalent to:

```
- auto alloc = std::move(this->alloc);
-
- allocator_traits<alloc-t>::destroy(alloc, this);
- allocator_traits<alloc-t>::deallocate(alloc, this, 1);

+ using traits = allocator_traits<Alloc>::template
+   rebind_traits<spawn-state>;
+ typename traits::allocator_type alloc(std::move(this->
+   alloc));
+ traits::destroy(alloc, this);
+ traits::deallocate(alloc, this, 1);
```

11 The expression `spawn(sndr, token, env)` is of type `void` and has the following effects:

- (11.1) — Uses `alloc` to allocate and construct an object `o` of type `that is a specialization of spawn-state` `decltype(spawn-state(alloc, write_env(token.wrap(sndr), senv), token))` from `alloc`, `write_env(token.wrap(sndr), senv)`, and `token` and then invokes `o.run()`. If an exception is thrown then any constructed objects are destroyed and any allocated memory is deallocated.

§ 2.9 US 221-339

[Editor's note: This wording is relative to the working draft after the application of the resolution of US 209-232.]

Edit 33.9.12.11 [\[exec.bulk\]](#) as indicated:

- 5 The exposition-only class template `impls-for` (33.9.2 [\[exec.snd.expos\]](#)) is specialized for `bulk_chunked_t` as follows:

```
namespace std::execution {
    template<>
    struct impls-for<bulk_chunked_t> : default-impls {
        static constexpr auto complete = see below;

        template<class Sndr, class... Env>
        static consteval void check-types();
    };
}
```

The member `impls-for<bulk_chunked_t>::complete` is initialized with a callable object equivalent to the following lambda:

[...]

```
template<class Sndr, class... Env>
    static consteval void check-types();

? Effects: Equivalent to:

    auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-
        T(Env)...>();
    auto fn = [<class... Ts>(set_value_t*)(Ts...)] {
        using data_type = data-type<Sndr>;
        if constexpr (!invocable<remove_cvref_t<child-type<data_type>&,
            remove_cvref_t<data-type<data_type>>,
            Ts&...>)
            throw unspecified-exception();
    };
    cs.for-each(overload-set(fn, [](auto){}));
```

- 6 The exposition-only class template `impls-for` (33.9.2 [\[exec.snd.expos\]](#)) is specialized for `bulk_unchunked_t` as follows:

```
namespace std::execution {
    template<>
    struct impls-for<bulk_unchunked_t> : default-impls {
        static constexpr auto complete = see below;

        template<class Sndr, class... Env>
```

```
static constexpr void check-types();
};
}
```

The member `impls-for<bulk_unchained_t>::complete` is initialized with a callable object equivalent to the following lambda:

[...]

```
template<class Sndr, class... Env>
static constexpr void check-types();
```

⁷ *Effects:* Equivalent to:

```
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-
T(Env)...>();
auto fn = [<class... Ts>(set_value_t*)(Ts...)] {
    using data_type = data-type<Sndr>;
    if constexpr (!invocable<remove_cvref_t<child-type<data_type>>&,
                          remove_cvref_t<data-type<data_type>Sndr>>,
                          Ts&...>)
        throw unspecified-exception();
};
cs.for-each(overload-set(fn, [(auto){})));
```

§ 2.10 [US 225-341](#)

1. Edit 22.4.4.1 [\[tuple.tuple.general\]](#) as indicated:

```
namespace std {
    template<class... Types>
    class tuple {
    public:
        // [tuple.cnstr], tuple construction
        constexpr explicit(see below) tuple();
        constexpr explicit(see below) tuple(const Types&...) noexcept(see below);
        // only if sizeof...(Types) >= 1
        template<class... UTypes>
            constexpr explicit(see below) tuple(UTypes&&...) noexcept(see below);
        // only if sizeof...(Types) >= 1

        tuple(const tuple&) = default;
        tuple(tuple&&) = default;

        template<class... UTypes>
            constexpr explicit(see below) tuple(tuple<UTypes...>&);
        template<class... UTypes>
            constexpr explicit(see below) tuple(const tuple<UTypes...>&);
        template<class... UTypes>
            constexpr explicit(see below) tuple(tuple<UTypes...>&&);
        template<class... UTypes>
            constexpr explicit(see below) tuple(const tuple<UTypes...>&&);

        template<class U1, class U2>
            constexpr explicit(see below) tuple(pair<U1, U2>&);           // only if
            sizeof...(Types) == 2
        template<class U1, class U2>
```

```

    constexpr explicit(see below) tuple(const pair<U1, U2>&); // only if
        sizeof...(Types) == 2
template<class U1, class U2>
    constexpr explicit(see below) tuple(pair<U1, U2>&&); // only if
        sizeof...(Types) == 2
template<class U1, class U2>
    constexpr explicit(see below) tuple(const pair<U1, U2>&&); // only if
        sizeof...(Types) == 2

template<tuple-like UTuple>
    constexpr explicit(see below) tuple(UTuple&&);

// allocator-extended constructors
template<class Alloc>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a);
template<class Alloc>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const Types&...);
template<class Alloc, class... UTypes>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
template<class Alloc>
    constexpr tuple(allocator_arg_t, const Alloc& a, const tuple&);
template<class Alloc>
    constexpr tuple(allocator_arg_t, const Alloc& a, tuple&&);
template<class Alloc, class... UTypes>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&);
template<class Alloc, class... UTypes>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
template<class Alloc, class... UTypes>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
template<class Alloc, class... UTypes>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&&);
template<class Alloc, class U1, class U2>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&);
template<class Alloc, class U1, class U2>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
template<class Alloc, class U1, class U2>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
template<class Alloc, class U1, class U2>
    constexpr explicit(see below)
        tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&&);

template<class Alloc, tuple-like UTuple>
    constexpr explicit(see below) tuple(allocator_arg_t, const Alloc& a,
        UTuple&&);

// [tuple.assign], tuple assignment
constexpr tuple& operator=(const tuple&);
constexpr const tuple& operator=(const tuple&) const;
constexpr tuple& operator=(tuple&&) noexcept(see below);

```

```

constexpr const tuple& operator=(tuple&&) const;

template<class... UTypes>
constexpr tuple& operator=(const tuple<UTypes...>&);
template<class... UTypes>
constexpr const tuple& operator=(const tuple<UTypes...>&) const;
template<class... UTypes>
constexpr tuple& operator=(tuple<UTypes...>&&);
template<class... UTypes>
constexpr const tuple& operator=(tuple<UTypes...>&&) const;

template<class U1, class U2>
constexpr tuple& operator=(const pair<U1, U2>&);           // only if
    sizeof...(Types) == 2
template<class U1, class U2>
constexpr const tuple& operator=(const pair<U1, U2>&) const;           // only if
    sizeof...(Types) == 2
template<class U1, class U2>
constexpr tuple& operator=(pair<U1, U2>&&);           // only if
    sizeof...(Types) == 2
template<class U1, class U2>
constexpr const tuple& operator=(pair<U1, U2>&&) const;           // only if
    sizeof...(Types) == 2

template<tuple-like UTuple>
constexpr tuple& operator=(UTuple&&);
template<tuple-like UTuple>
constexpr const tuple& operator=(UTuple&&) const;

// [tuple.swap], tuple swap
constexpr void swap(tuple&) noexcept(see below);
constexpr void swap(const tuple&) const noexcept(see below);
};

template<class... UTypes>
tuple(UTypes...) -> tuple<UTypes...>;
template<class T1, class T2>
tuple(pair<T1, T2>) -> tuple<T1, T2>;
template<class Alloc, class... UTypes>
tuple(allocator_arg_t, Alloc, UTypes...) -> tuple<UTypes...>;
template<class Alloc, class T1, class T2>
tuple(allocator_arg_t, Alloc, pair<T1, T2>) -> tuple<T1, T2>;
template<class Alloc, class... UTypes>
tuple(allocator_arg_t, Alloc, tuple<UTypes...>) -> tuple<UTypes...>;
}

```

2. Edit 22.4.4.2 [\[tuple.cnstr\]](#) as indicated:

```

constexpr explicit(see below) tuple(const Types&...)
    noexcept((is_nothrow_copy_constructible_v<Types> && ...));

```

⁹ *Constraints:* [...]

¹⁰ *Effects:* [...]

¹¹ *Remarks:* [...]

```
template<class... UTypes>
constexpr explicit(see below) tuple(UTypes&&... u)
    noexcept((is_nothrow_constructible_v<Types, UTypes> && ...));
```

12 [...]

13 *Constraints:* [...]

14 *Effects:* [...]

15 *Remarks:* [...]

§ 3 References

[N5014] Thomas Köppe. 2025-08-05. Working Draft, Standard for Programming Language C++. <https://wg21.link/n5014>